



**SCHOOL OF
COMPUTING**

Design Analysis of Algorithm

CSE 211

Week-2

CH.SC.U4CSE24221

TEJESH KANTA

Bubble Sort

Code:

```
#include <stdio.h>

int main() {
    int arr[100], n, i, j, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // bubble sort
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }

    printf("Sorted array:\n");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
amma@amma18:~$ cd Desktop
amma@amma18:~/Desktop$ gcc bubble1.c -o bubble1
amma@amma18:~/Desktop$ ./bubble1
Enter number of elements: 5
Enter the elements:
12 3 4 17 35
Sorted array:
3 4 12 17 35 amma@amma18:~/Desktop$
```

Bubble Sort

Code:

Space Complexity and it's Justification:

Time Complexity

- Best case: $O(n^2)$
→ No optimization (no early break used).
- Average case: $O(n^2)$
- Worst case: $O(n^2)$
→ Nested loops compare all adjacent pairs.

Space Complexity

- $O(1)$
→ Only one temporary variable used (in-place sorting).

Insertion Sort

Code:

```
#include <stdio.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n, i;

    printf("CH.SC.U4CSE24221-KANTA TEJESH\n");

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    insertionSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

OUTPUT:

```
amma@amma18:~/Desktop$ gcc insertion.c -o insertion
amma@amma18:~/Desktop$ ./insertion
CH.SC.U4CSE24221-KANTA TEJESH
Enter number of elements: 7
Enter 7 elements:
15 14 16 10 18 35 23
Sorted array: 10 14 15 16 18 23 35 amma@amma18:~/Desktop$
```

Insertion Sort

Code:

Space Complexity and it's Justification:

Time Complexity

- Best: $O(n)$
Justification:
 - Already sorted array → only one comparison per element
- Average / Worst: $O(n^2)$
Justification:
 - Each element may shift through the entire sorted part

Space Complexity

- $O(1)$
Justification:
 - Uses only one temporary variable (key)

Selection Sort

Code:

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;

    for (i = 0; i < n - 1; i++) {
        minIndex = i;

        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int n, i;
    printf("CH.SC.U4CSE24221- KANTA TEJESH \n");
    printf("Enter number of elements: ");
    scanf("%d", &n);
    |
    int arr[n];

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    selectionSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Selection Sort

Code:

```
amma@amma18:~$ cd Desktop
amma@amma18:~/Desktop$ gcc selection.c -o selection
amma@amma18:~/Desktop$ ./selection
CH.SC.U4CSE24221- KANTA TEJESH
Enter number of elements: 5
Enter 5 elements:
12 65 43 32 54
Sorted array: 12 32 43 54 65 amma@amma18:~/Desktop$
```

Space Complexity and it's Justification:

Time Complexity

- Best / Average / Worst: $O(n^2)$

Justification:

- Always compares every element to find minimum
- Number of comparisons does not depend on input order

Space Complexity

- $O(1)$

Justification:

- Sorting done **in-place**, no extra memory used

Bucket Sort

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n, i, j;

    printf("CH.SC.U4CSE24221-KANTA TEJESH\n");

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}
```


Bucket Sort

Code:

```
int max = arr[0];
for (i = 1; i < n; i++) {
    if (arr[i] > max)
        max = arr[i];
}

int buckets[MAX][n];
int bucketCount[MAX];

for (i = 0; i < MAX; i++)
    bucketCount[i] = 0;

for (i = 0; i < n; i++) {
    int bucketIndex = (arr[i] * MAX) / (max + 1);
    buckets[bucketIndex][bucketCount[bucketIndex]++] = arr[i];
}

for (i = 0; i < MAX; i++) {
    if (bucketCount[i] > 0)
        insertionSort(buckets[i], bucketCount[i]);
}

int index = 0;
for (i = 0; i < MAX; i++) {
    for (j = 0; j < bucketCount[i]; j++) {
        arr[index++] = buckets[i][j];
    }
}

// Print sorted array
printf("Sorted array: ");
for (i = 0; i < n; i++)
```

```
    printf("%d ", arr[i]);
```

```
    return 0;
```

```
}
```

OUTPUT:

```
amma@amma18:~/Desktop$ ./bucket
CH.SC.U4CSE24221-KANTA TEJESH
Enter number of elements: 5
Enter 5 elements:
9 8 10 6 3
Sorted array: 3 6 8 9 10 amma@amma18:~/Desktop$
```

Space Complexity and it's Justification:

Time Complexity

- Best case: $O(n)$
→ Elements uniformly distributed, minimal shifting inside buckets.

Bucket Sort

Code:

- Average case: $O(n)$
→ Each bucket has few elements; insertion is fast.
- Worst case: $O(n^2)$
→ All elements fall into one bucket → insertion sort behavior.

Space Complexity

- $O(n)$
→ Buckets array `buckets[10][n]` + count array.

Heap Sort

Code:

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (i = n - 1; i >= 0; i--) {
        temp = arr[0];
```

Heap Sort

Code:

```
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

int main() {
    int n, i;

    printf("CH.SC.U4CSE24221-KANTA TEJESH\n");

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    heapSort(arr, n);

    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

OUTPUT:

```
CH.SC.U4CSE24221-KANTA TEJESH
Enter number of elements: 5
Enter 5 elements: 45 56 24 54 63
Sorted array: 24 45 54 56 63
```

Space Complexity and it's Justification:

Time Complexity

Heap Sort

Code:

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$
→ Heap construction $O(n)$ + n heapify operations $O(\log n)$.

Space Complexity

- $O(\log n)$
→ Due to recursive heapify() call stack.